

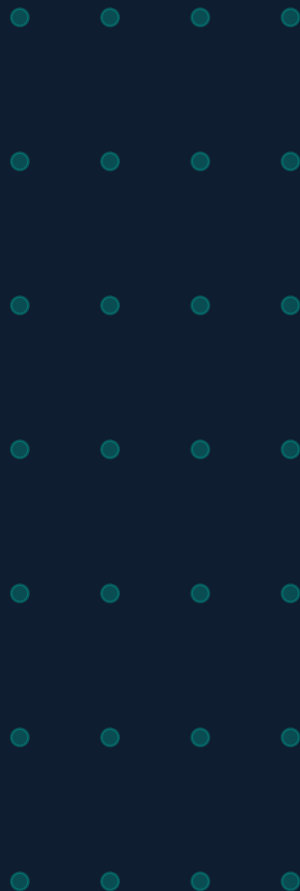
PROJECT REPORT

MyCobot 280Pi

ROS2-Based Robotic Manipulation
System Design & Implementation

Elephant Robotics . ROS2 . YOLOv11 . Python

Robotics Semester Project . 2025--2026



Outline

- **Introduction**
- **System Setup**
- **Visualization and Simulation**
- **mycobot_control node**
- **Pick & Place application**
- **Jupyter Notebooks**

01

Introduction

MyCobot 280Pi -- Hardware Overview & Context

What is the MyCobot 280Pi?

Manufactured by Elephant Robotics, the MyCobot 280Pi is a compact 6-DOF collaborative robotic arm designed for education and research.

6 DOF

Articulate arm

280 mm

Maximum reach

250 g

Max payload

Rpi 4B

Onboard computer



Why Rebuild the Control Stack?

The myCobot 280 Pi ships with a Python SDK optimized for guided demos, but limited in supporting modular, research-grade robotics workflows.

WHAT IT PROVIDES

Vendor SDK

- Direct Robot Control Via Python
- Basic Motion Execution
- Quick Teaching Demos

WHAT IT LACKS

Limitations for Research & Teaching

- Rigid Architecture
- Position Only
- Hardware Only
- Basic Motion Planning
- Black Boxed

Project Roadmap



01

SIMULATE

URDF + Gazebo + RViz



02

CONTROL

Custom ROS 2 Control Stack



03

APPLY

Vision-Driven Pick & Place



04

TEACH

Educational Toolkit

02

System Setup

Ubuntu & ROS2 -- Development Environment

Why This Stack?

THE CHOSEN PAIRING

Ubuntu 24.04 LTS + ROS 2 Jazzy

LTS ALIGNMENT

PYTHON 3.12 + ML
STACK

SIM + HARDWARE
NATIVE

03

Visualization and Simulation

Robot Model Creation and Simulation Package

Building a Simulation Ready Robot Model

NATIVE ELEPHANT ROBOTICS URDF

Limitations of Vendor URDF

- ✗ Meshes not resolving → robot not visible in RViz
- ✗ Missing inertial properties → incompatible with Gazebo
- ✗ Mesh based collision geometry → inefficient and unstable

OUR PATCHED URDF

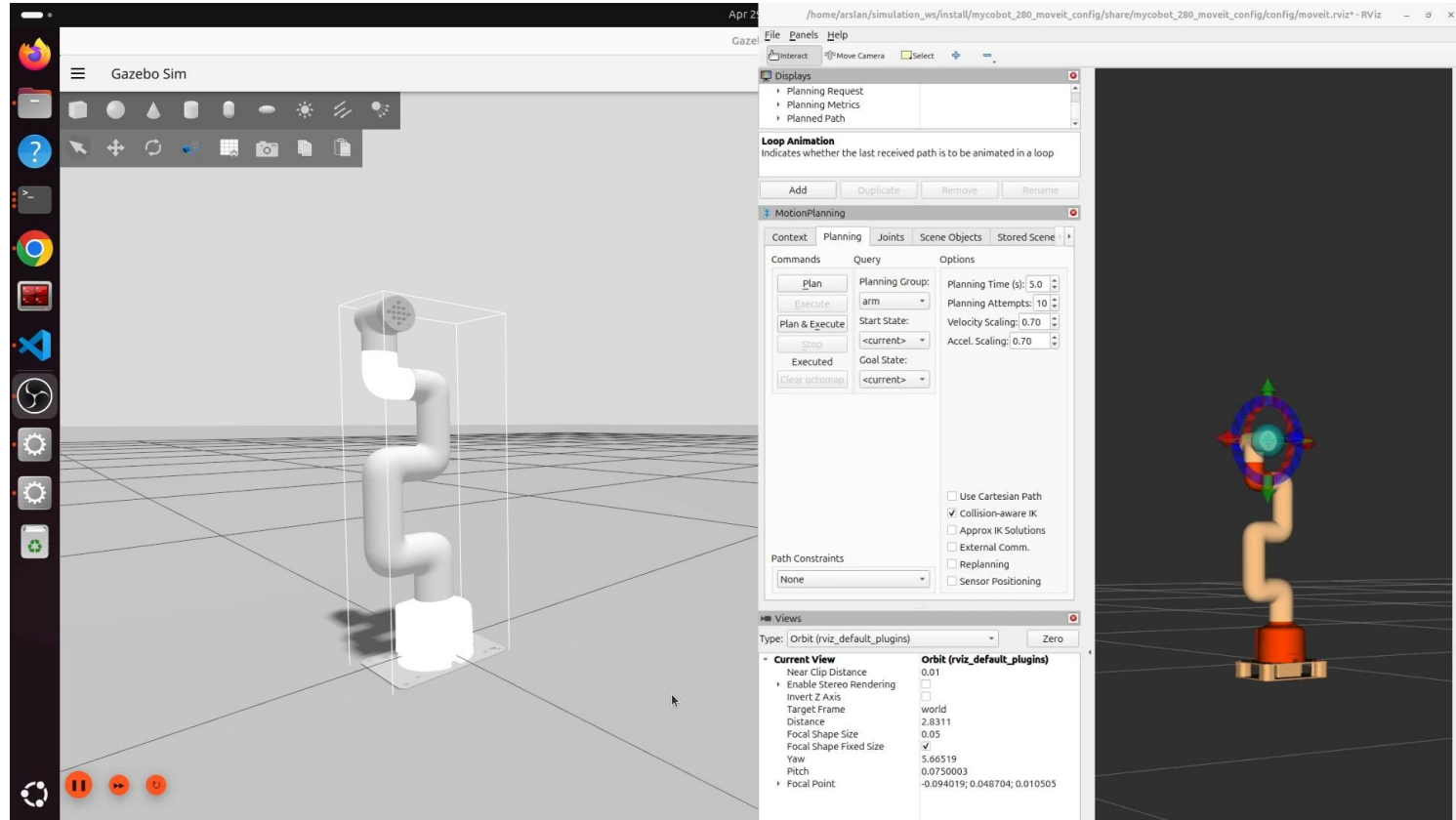
What We Added

- ✓ Corrected mesh paths and ensured full model visibility
- ✓ Added physically reasonable inertial parameters per link
- ✓ Replaced mesh-based collision geometry with simplified primitives

Gazebo Simulation Package



Trajectory Planning and Execution Pipeline



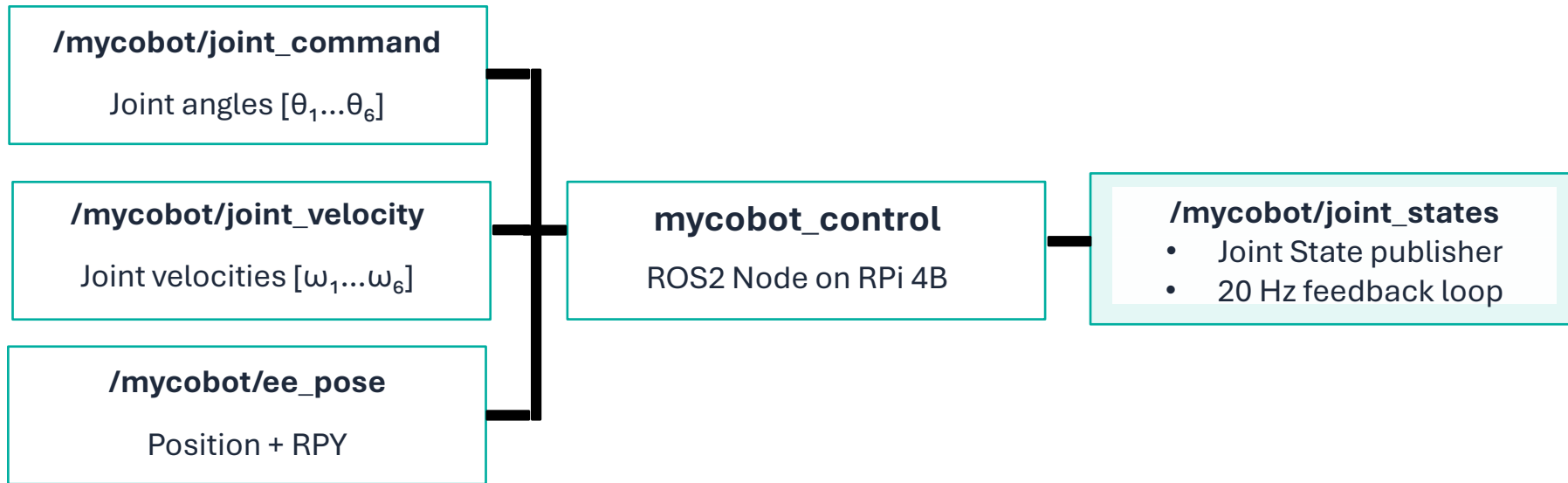
04

mycobot_control Node

Unified ROS2 Control -- Without pymycobot

Architecture Overview

A single, self-contained ROS2 node (mycobot_control.py) runs natively on the Raspberry Pi 4B. It replaces the pymycobot library with a custom serial protocol and exposes three unified control interfaces on standard ROS2 topics:



Joint Position Control

- Subscribing to /mycobot/joint_command, the node receives target joint angle arrays $[\theta_1 \dots \theta_6]$ in radians.
- A trapezoidal velocity profile generator ensures smooth, acceleration-limited motion to the target.

Trapezoidal Profiling

Commands are interpolated with ramp-up and ramp-down phases.

Ramp Rate Limiting

- Configurable max acceleration bounds $\Delta\text{velocity}/\Delta t$ per joint
- Prevents mechanical shock from large step commands.

Angle Clamping

Commanded angles are automatically clamped to each joints hard limits before serialization.

6-Joint Simultaneous

- All 6 joints commanded atomically at 20 Hz.
- Serial protocol packs all angles into a single frame for synchronised hardware execution.

Joint Velocity Control

Subscribing to /mycobot/joint_velocity (Float64MultiArray), the node accepts continuous velocity commands $[\omega_1 \dots \omega_6]$ in rad/s. This mode enables real-time teleoperation and incremental IK stepping.

Max Velocity

150 °/s

~2.618 rad/s -- configurable hard cap per joint

Control Rate

20 Hz

Commands integrated at 50 ms cycle time

Safety Stop

Joint Limits

Integration halts at $\pm$ limits -- no position overshoot

Protocol

Custom Serial

Raw angle increments via UART

Inverse Kinematics -- EE Pose Control

The /mycobot/ee_pose subscriber accepts geometry_msgs/Pose (xyz position + quaternion orientation). A custom Damped Least Squares (DLS) IK solver -- built directly into the node -- resolves joint angles without any external IK library.

1

Build FK Chain

At startup the node parses the URDF (robot_description parameter) and constructs the full FK chain.

2

Compute Jacobian

Geometric Jacobian $J(q)$ computed numerically at each iteration step using finite differences over the FK chain.

3

DLS Update Step

$\Delta q = J^T(JJ^T + \lambda^2 I)^{-1} \cdot \Delta x$ -- damping factor λ prevents singularity blow-up.

4

Convergence Check

Iteration halts when position error < 2 mm AND orientation error < 0.02 rad, or max iterations reached.

IK Solver -- Parameters & Configuration

XYZ

Position Tolerance

2 mm

RPY

Orientation Tolerance

0.02 rad

λ

Damping Factor λ

Configurable

N

Max Iterations

Configurable

Hz

Control Rate

20 Hz

Δq

Step Size

Configurable

- All IK parameters are implemented ROS2 node parameters -- tunable at launch time.
- The solver was integrated directly into mycobot_control to avoid any runtime dependencies.

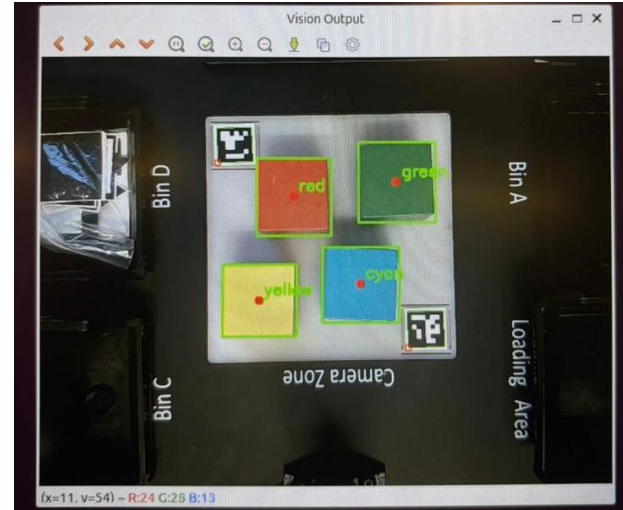
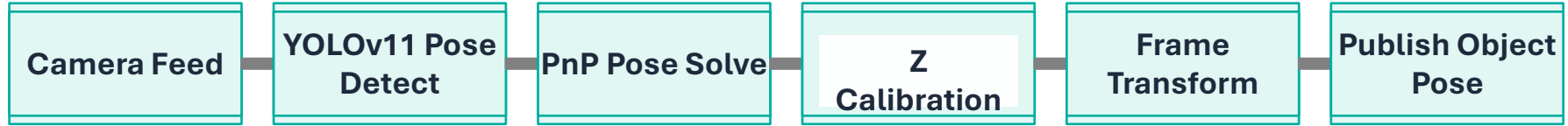
05

Pick & Place Application

Object Detection, Pose Estimation & Colour Sorting

System Overview

A YOLOv11 based vision pipeline which detects colored cubes in the robot's workspace, estimates their 6-DOF pose using PnP, and publishes structured object data for the pick-and-place motion node.



Model Training - I

Dataset

400 annotated images of coloured cubes

Each object labelled with 5 key-points (4 corners + centre)

Train/Validation split: 80/20

Model & Configuration

YOLOv11 Pose model tuned for cube detection

Input image size: 640 × 640

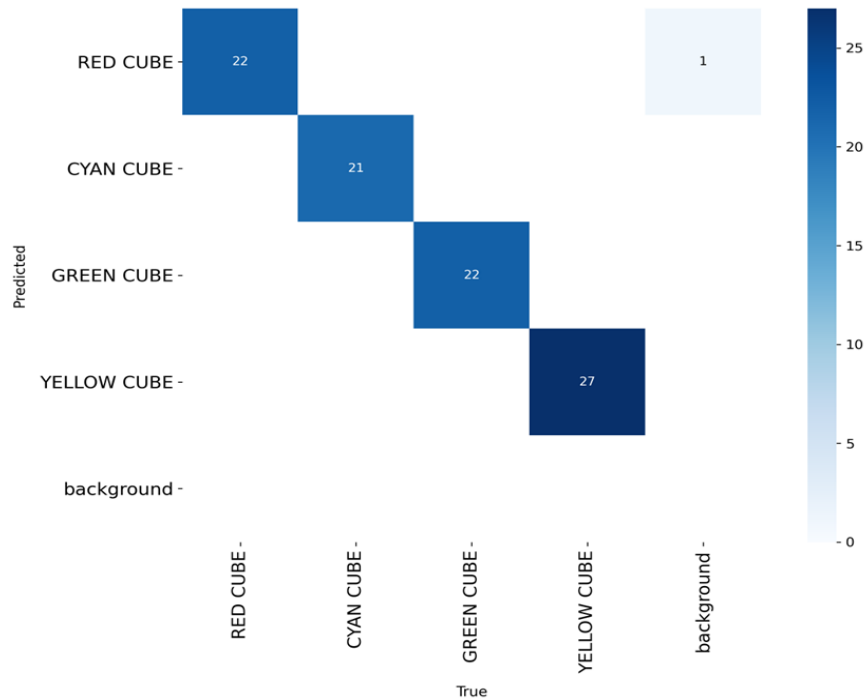
Designed for key-point-based pose estimation

Training Setup

Epochs: 100

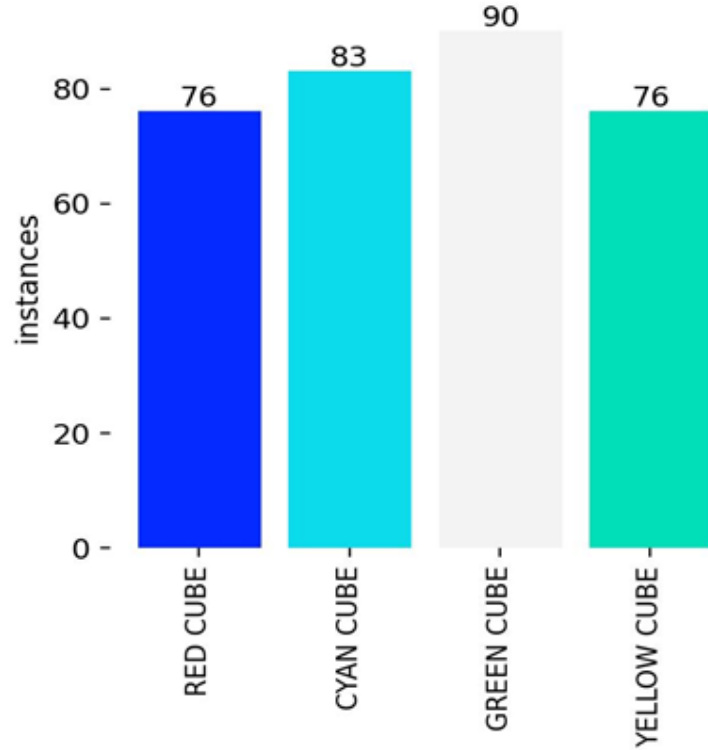
Batch size: 4

GPU-based training

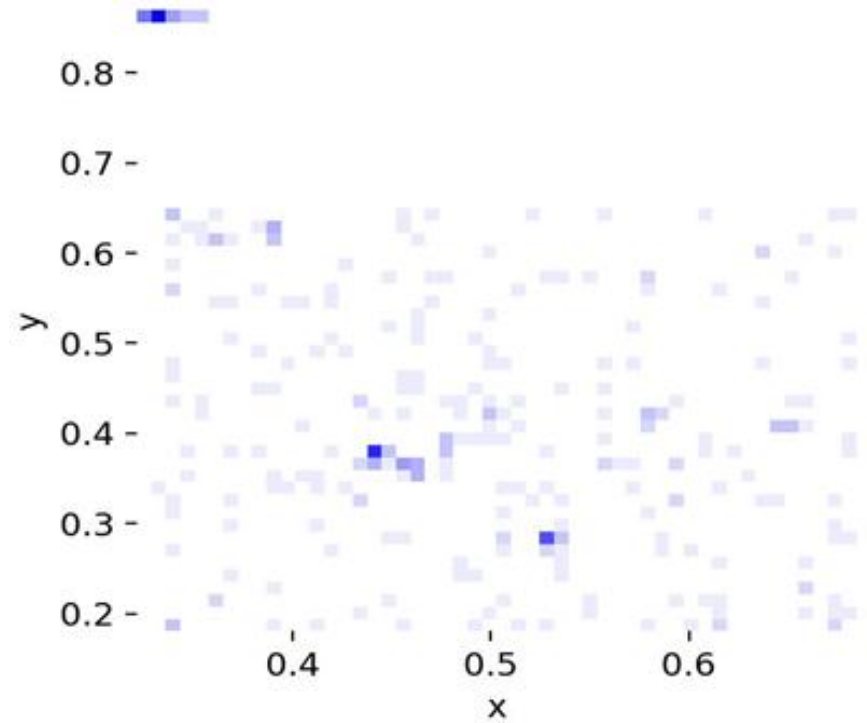


Confusion Matrix

Model Training - II



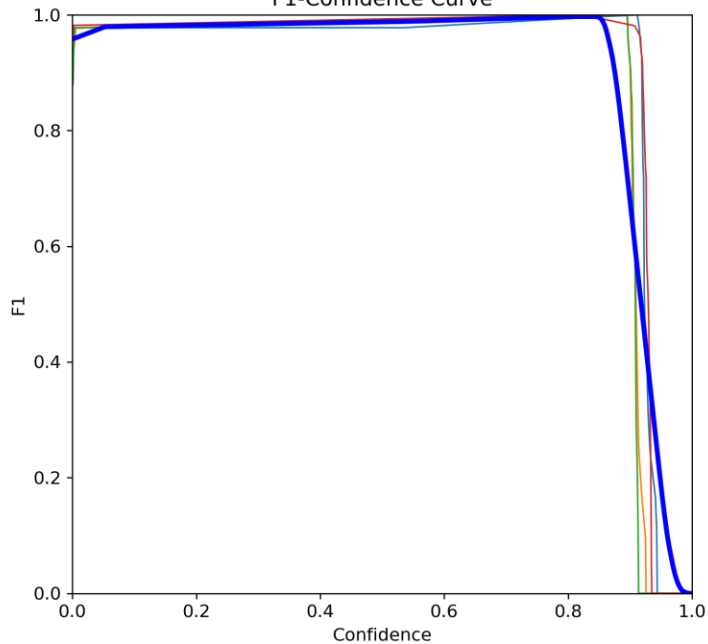
Class Distribution



Centre Position Distribution

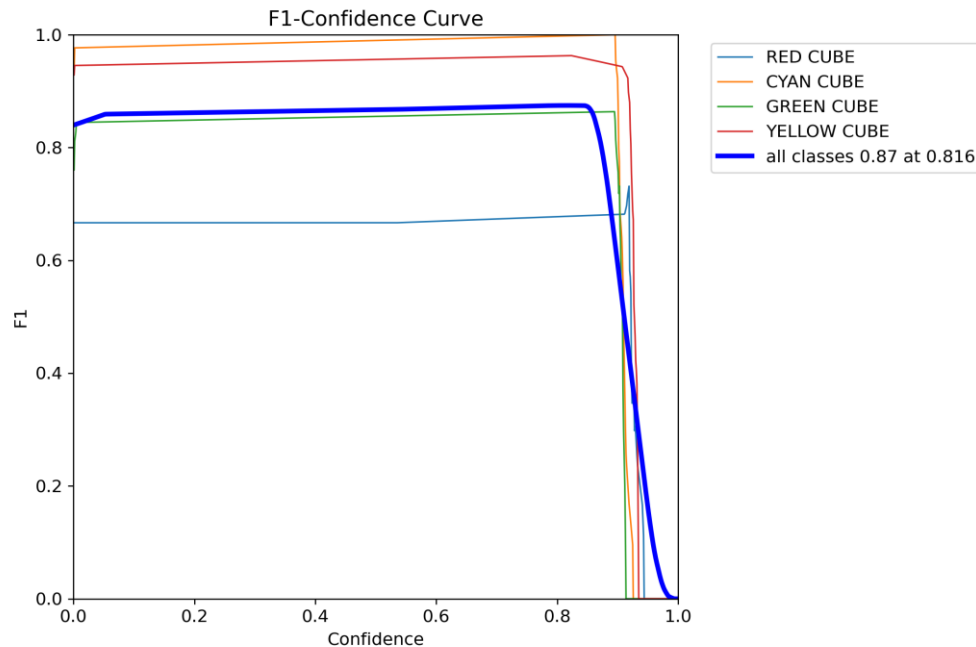
Model Performance - I

F1-Confidence Curve

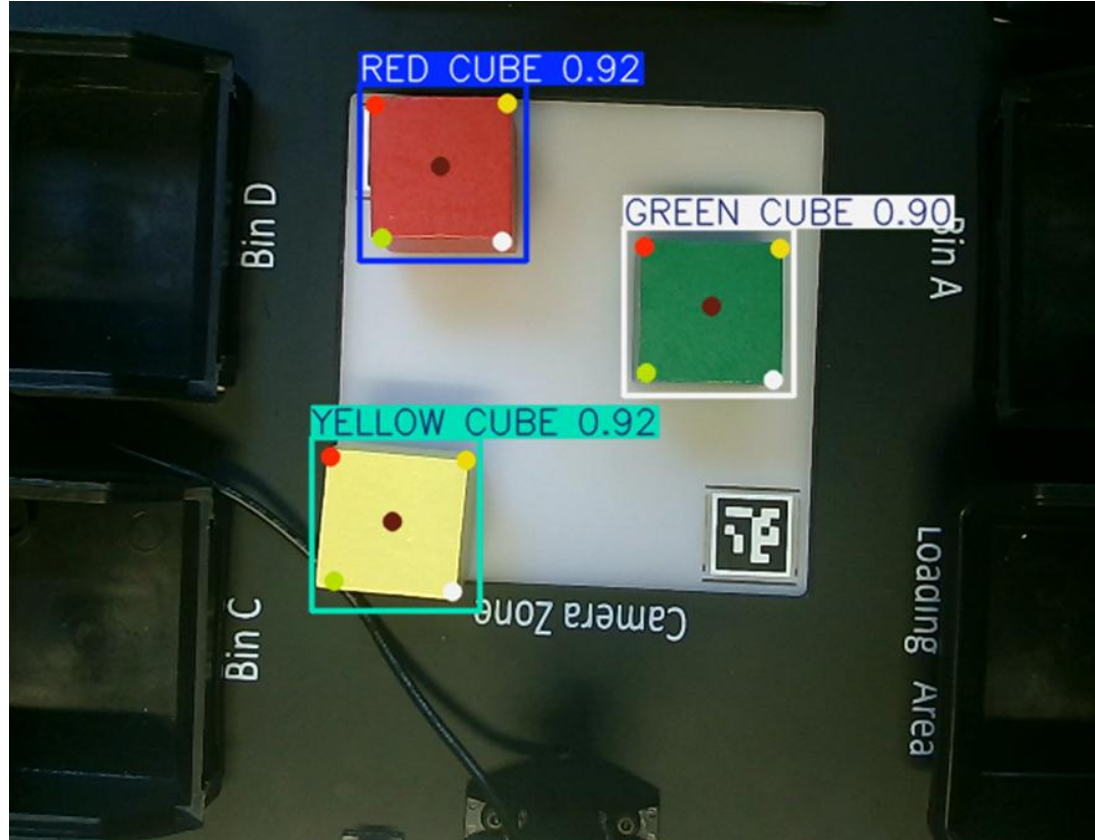


Box Layout Confidence

Pose Confidence

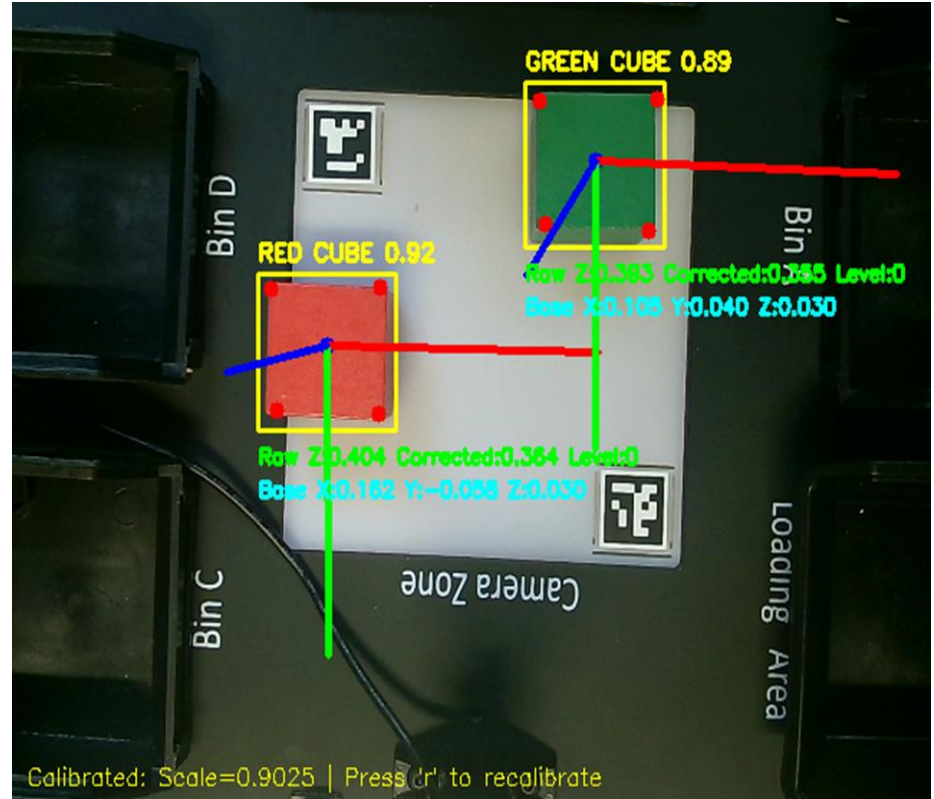


Model Performance - II



Cube Detection & Processing

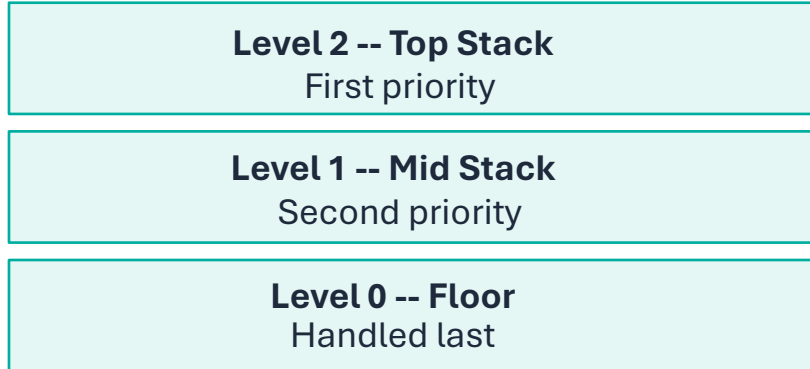
- **Detection:**
YOLOv11 detects cubes with bounding boxes and key-points
- **Pose Estimation & Calibration:**
PnP computes 3D pose; depth corrected using calibration
- **Stack Detection:**
Calibrated Z compared with cube height to determine stack level
- **ROS2 Publishing:**
Publish color and (x, y, z) to
'/detected_objects' for robot control



Motion Pipeline and Sorting



Priority 1 -- Stack Level



Priority 2 -- Color



06

Jupyter Notebooks

Educational Lab Series for Future Students

Jupyter Notebooks — Educational Lab Series

- 8 lab sets: Template + Solution + PDF Manual
- Structured curriculum: basic mathematics → full pick-and-place
- Mathematical Foundations
- Rotations, transforms, quaternions, SLERP — built with NumPy
- Kinematics & Control
- FK, IK, Jacobian, trajectory planning for a real 6-DOF arm
- Hardware Integration
- Serial protocol, ROS 2 topics, TCP calibration, GPIO pump
- Capstone
- Vision-driven pick-and-place with colour sorting

Lab Series — Theory & Kinematics (Labs 1–4)

- From mathematical primitives to velocity-level kinematics
- Lab 1 — Spatial Transformations
 - Rotation matrices, Euler/RPY, axis-angle, quaternions, SLERP
- Lab 2 — Forward Kinematics
 - DH convention, URDF chain, Robotics Toolbox, ROS 2 / RViz2
- Lab 3 — Inverse Kinematics
 - Damped least-squares, geometric seeds, convergence debugging
- Lab 4 — Differential Kinematics
 - 6×6 Jacobian, singularity analysis, manipulability ellipsoid

Lab Series — Practical Robotics (Labs 5–8)

- From calibration to autonomous manipulation
- Lab 5 — TCP Calibration
- Flange-to-TCP estimation, SVD mean rotation, residual validation
- Lab 6 — Trajectory Planning
- Trapezoidal profiles, min-time motion, codebase synchronization
- Lab 7 — Serial Communication
- ATOM frame protocol, position/velocity mode, safety stops, ROS 2
- Lab 8 — Pick and Place
- Vision pipeline, Cartesian commands, GPIO pump, colour-based sorting

Template & Solution — Design Philosophy

▪ Template Notebook

- ✓ Theory + equations in markdown
- ✓ Code cells with ... placeholders
- ✓ Helper functions provided
- ✓ ROS 2 integration stubs
- ✓ Safety flags (motion disabled)

▪ Solution Notebook

- ✓ Complete working code
- ✓ Codebase-aligned implementations
- ✓ Validation checks and plots
- ✓ Solved code ready to execute
- ✓ Motion disabled by default

Recommendations & Next Steps

Improve IK Robustness

Workspace reachability checks, null-space optimization, and singularity avoidance near joint limits.

Expand YOLO Training Data

Retrain on additional cube colors and shapes with varied lighting and orientation data for better results.

Extend Jupyter Curriculum

Add labs on vision-based control, and reinforcement learning policies running on the physical MyCobot 280Pi.